

Computer Graphics Mini Project CS3102

Mini Project 3D Plane Crash Game

Submitted by

Renu Bojja 1RVU22CSE129

Karmishtha Patnaik 1RVU22CSE077

School of Computer Science

RV University

.....

Submitted to

Prof. Evlin Vidya Latha P

Professor

School of Computer Science

RV University

Problem Statement

Creating realistic and interactive 3D games involves combining multiple computer graphics techniques, including object transformations, real-time rendering, user interaction, and physics simulation. However, developing such a system that runs smoothly while maintaining visual appeal and accurate flight mechanics is a challenging task.

This project aims to address the challenge by designing and implementing a **3D Plane Crash Game using OpenGL**, where players control an aircraft in a dynamic environment filled with obstacles. The system must handle object transformations, collision detection, and realistic visual effects during crashes, while also supporting user interactivity through real-time keyboard and mouse input. Ensuring a smooth, immersive experience will require the integration of techniques such as double buffering, lighting and shading and basic camera manipulation.

Implementation

This OpenGL-based 3D flight game starts with a start screen, transitions into gameplay, and ends with a game over screen based on player interaction. The airplane, rendered using scaled cubes, is the main player-controlled object that flies forward in a 3D space. Obstacles in the form of rings and buildings are generated and move toward the player to simulate forward motion. The game uses camera following, lighting, and proper matrix stack management (glPushMatrix/glPopMatrix) to render dynamic scenes. The player navigates through rings and avoids buildings, and the score increases when successfully flying through rings. Game state transitions and heads-up display (HUD) elements like text and score are handled using GLUT functions.

OpenGL Functions in the Code

1. glut/GL Functions:

- glutInit(&argc, argv): Initializes the GLUT library
- glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH): Sets display mode with double buffering, RGB color, and depth buffer
- glutInitWindowSize(800, 600): Sets initial window size
- glutCreateWindow("Flight Through Rings - OpenGL Game"): Creates a window with the specified title
- glutDisplayFunc(display): Registers display callback function
- glutReshapeFunc(reshape): Registers reshape callback for window resize events
- glutPassiveMotionFunc(mouseMotion): Registers callback for mouse movement
- glutTimerFunc(25, update, 0): Sets a timer to call update function after 25ms
- glutKeyboardFunc(keyPressed): Registers keyboard press callback
- glutKeyboardUpFunc(keyReleased): Registers keyboard release callback

- `glutSwapBuffers()`: Swaps front and back buffers for double buffering
- `glutPostRedisplay()`: Marks window for redrawing on next iteration
- `glutMainLoop()`: Enters the GLUT event processing loop
- `glutSolidCube(1.0f)`: Renders a solid cube
- `glutSolidTorus(0.15f, obstacles[i].radius, 30, 30)`: Renders a solid torus (ring)
- `glutBitmapCharacter()`: Renders a character from a bitmap font
- `glutWarpPointer(mouseX, mouseY)`: Moves the cursor to specified position
- `glutSetCursor(GLUT_CURSOR_NONE/INHERIT)`: Sets cursor visibility

2. GL State Functions:

- `glEnable(GL_DEPTH_TEST)`: Enables depth testing
- `glEnable(GL_LIGHTING)`: Enables lighting calculations
- `glEnable(GL_LIGHT0)`: Enables light source 0
- `glEnable(GL_COLOR_MATERIAL)`: Enables material color tracking
- `glEnable(GL_NORMALIZE)`: Enables automatic normalization of normals
- `glDisable(GL_COLOR_MATERIAL)`: Disables material color tracking
- `glDisable(GL_LIGHTING)`: Disables lighting calculations
- `glClearColor(0.0f, 0.0f, 0.0f, 1.0f)`: Sets clear color to black

3. GL Matrix Operations:

- `glMatrixMode(GL_PROJECTION)`: Sets current matrix to projection matrix
- `glMatrixMode(GL_MODELVIEW)`: Sets current matrix to modelview matrix
- `glLoadIdentity()`: Replaces current matrix with identity matrix
- `glPushMatrix()`: Pushes current matrix onto stack
- `glPopMatrix()`: Pops matrix off stack, restoring previous matrix
- `glOrtho(0.0, 800, 0.0, 600, -1.0, 1.0)`: Sets orthographic projection
- `gluPerspective(45.0f, 800.0f/600.0f, 0.1f, 200.0f)`: Sets perspective projection
- `gluLookAt()`: Creates viewing matrix for camera position

4. GL Transformations:

- `glTranslatef(x, y, z)`: Translates by (x,y,z)
- `glRotatef(angle, x, y, z)`: Rotates by angle degrees around (x,y,z) axis
- `glScalef(x, y, z)`: Scales by factors (x,y,z)

5. GL Drawing Commands:

- `glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)`: Clears color and depth buffers
- `glBegin(GL_TRIANGLES)`: Begins drawing triangles
- `glBegin(GL_QUADS)`: Begins drawing quadrilaterals
- `glBegin(GL_LINE_LOOP)`: Begins drawing connected line segments
- `glVertex3f(x, y, z)`: Specifies a vertex at (x,y,z)
- `glVertex2f(x, y)`: Specifies a vertex at (x,y) with z=0
- `glEnd()`: Ends drawing primitive
- `glRasterPos2i(x, y)`: Sets current raster position

- `glViewport(0, 0, w, h)`: Sets viewport dimensions

6. GL Property Settings:

- `glColor3f(r, g, b)`: Sets current color
- `glLightfv(GL_LIGHT0, GL_POSITION, lightPos)`: Sets light position
- `glLightfv(GL_LIGHT0, GL_AMBIENT, ambient)`: Sets ambient light properties
- `glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuse)`: Sets diffuse light properties
- `glColorMaterial(GL_FRONT, GL_AMBIENT_AND_DIFFUSE)`: Sets which material properties track current color

1. Window and Application Setup (main function)

Initialization

- `glutInit(&argc, argv)`: Initializes the GLUT library, processing command line arguments
- `glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH)`: Configures display with:
 - `GLUT_DOUBLE`: Double buffering for smooth animation
 - `GLUT_RGB`: Standard RGB color model
 - `GLUT_DEPTH`: Depth buffer for 3D rendering
- `glutInitWindowSize(800, 600)`: Sets initial window dimensions to 800x600 pixels
- `glutCreateWindow("Flight Through Rings - OpenGL Game")`: Creates window with title

Callback Registration

- `glutDisplayFunc(display)`: Registers the function called when window needs redrawing
- `glutReshapeFunc(reshape)`: Registers function called when window is resized
- `glutPassiveMotionFunc(mouseMotion)`: Registers function for mouse movement without buttons pressed
- `glutTimerFunc(25, update, 0)`: Sets timer to call update function every 25ms (~40 FPS)
- `glutKeyboardFunc(keyPressed)`: Registers function for keyboard key press events
- `glutKeyboardUpFunc(keyReleased)`: Registers function for keyboard key release events

OpenGL State Configuration

- `glEnable(GL_DEPTH_TEST)`: Enables depth testing for proper 3D rendering
- `glClearColor(0.0f, 0.0f, 0.0f, 1.0f)`: Sets background color to black with full opacity

Lighting Setup

- `glEnable(GL_LIGHTING)`: Enables OpenGL lighting calculations
- `glEnable(GL_LIGHT0)`: Activates light source 0 (the default light)
- `glLightfv(GL_LIGHT0, GL_AMBIENT, ambient)`: Sets ambient light properties
- `glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuse)`: Sets diffuse light properties
- `glEnable(GL_NORMALIZE)`: Ensures normals are normalized for proper lighting

Projection Setup

- `glMatrixMode(GL_PROJECTION)`: Switches to projection matrix for setting up camera
- `glLoadIdentity()`: Clears current matrix to identity matrix
- `gluPerspective(45.0f, 800.0f/600.0f, 0.1f, 200.0f)`: Sets 45° field of view, aspect ratio, near and far clipping planes
- `glMatrixMode(GL_MODELVIEW)`: Switches back to modelview matrix for object transformations

Cursor Handling

- `glutSetCursor(GLUT_CURSOR_NONE)`: Hides cursor for immersive gameplay

Main Loop

- `glutMainLoop()`: Enters GLUT's event processing loop (never returns)

2. Display Function

- `glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)`: Clears color and depth buffers
- `glLoadIdentity()`: Resets the current matrix to identity
- `gluLookAt()`: Sets camera position, look-at point, and up vector
- `glutSwapBuffers()`: Swaps the front and back buffers

3. Plane Drawing (drawPlane function)

- `glPushMatrix()`: Saves current transformation matrix

- `glTranslatef(planeX, planeY, -5.0f)`: Positions plane in 3D space
- `glRotatef(roll, 0, 0, 1)`: Applies roll rotation (around z-axis)
- `glRotatef(pitch, 1, 0, 0)`: Applies pitch rotation (around x-axis)
- `glRotatef(90, 0, 1, 0)`: Rotates plane 90° around y-axis
- `glScalef(0.8f, 0.8f, 0.8f)`: Scales plane to 80% size
- `glColor3f(1.0f, 1.0f, 1.0f)`: Sets plane color to white

Geometry Drawing

- `glBegin(GL_TRIANGLES)`: Begins drawing triangles for nose, bottom, center crease, tail fin
- `glVertex3f(x, y, z)`: Specifies each vertex in 3D space
- `glEnd()`: Ends primitive drawing
- `glBegin(GL_QUADS)`: Begins drawing quadrilaterals for wings
- `glVertex3f(x, y, z)`: Specifies each vertex
- `glEnd()`: Ends primitive drawing
- `glPopMatrix()`: Restores previously saved transformation matrix

4. Obstacle Drawing (drawObstacles function)

- `glEnable(GL_LIGHTING)`: Enables lighting for obstacles
- `glEnable(GL_LIGHT0)`: Enables light source 0
- `glLightfv(GL_LIGHT0, GL_POSITION, lightPos)`: Sets light position
- `glEnable(GL_COLOR_MATERIAL)`: Allows color to affect material properties
- `glColorMaterial(GL_FRONT, GL_AMBIENT_AND_DIFFUSE)`: Sets which material properties track current color

For Each Obstacle

- `glPushMatrix()`: Saves current matrix
- `glTranslatef(obstacles[i].x, obstacles[i].y, obstacles[i].z)`: Positions obstacle
- `glRotatef(obstacles[i].rotation, 0.0f, 0.0f, 1.0f)`: Rotates obstacle
- `glPushMatrix()`: Saves matrix for spokes
- `glRotatef(j * 30.0f, 0.0f, 0.0f, 1.0f)`: Rotates each spoke
- `glColor3f(j % 2, (j + 1) % 2, 0.0f)`: Alternates colors
- `glutSolidTorus(0.15f, obstacles[i].radius, 30, 30)`: Draws torus with inner radius 0.15f, outer radius from obstacle, 30 sides and rings
- `glPopMatrix()`: Restores matrix after each spoke

- `glPopMatrix()`: Restores matrix after each obstacle
- `glDisable(GL_COLOR_MATERIAL)`: Disables color material tracking
- `glDisable(GL_LIGHTING)`: Disables lighting

5. Building Drawing (`drawBuildings` function)

- `glPushMatrix()`: Saves current matrix
- `glTranslatef(x, y, z)`: Positions each building
- `glEnable(GL_COLOR_MATERIAL)`: Enables color material
- `glColorMaterial(GL_FRONT, GL_AMBIENT_AND_DIFFUSE)`: Sets material properties to track color
- `glColor3f(r, g, b)`: Sets color for building from stored RGB values

Main Building Structure

- `glPushMatrix()`: Saves matrix
- `glScalef(width, height, width)`: Scales building
- `glutSolidCube(1.0f)`: Draws a solid cube
- `glPopMatrix()`: Restores matrix

Windows

- `glColor3f(0.1f, 0.1f, 0.3f)`: Sets dark blue/black for windows
- `glPushMatrix()`: Saves matrix for each window
- `glTranslatef(winX, winY, winZ)`: Positions each window
- `glScalef(windowSize, windowSize, 0.01f)`: Scales window
- `glutSolidCube(1.0f)`: Draws window as thin cube
- `glPopMatrix()`: Restores matrix after each window
- `glDisable(GL_COLOR_MATERIAL)`: Disables color material
- `glPopMatrix()`: Restores matrix after each building

6. Runway Drawing (`drawRunway` function)

- `glPushMatrix()`: Saves matrix
- `glTranslatef(0.0f, groundLevel - runwayThickness/2, 0.0f)`: Positions runway
- `glScalef(runwayWidth, runwayThickness, runwayLength)`: Scales runway
- `glColor3f(0.2f, 0.2f, 0.2f)`: Sets dark gray color
- `glutSolidCube(1.0f)`: Draws runway as flat cube
- `glPopMatrix()`: Restores matrix

Runway Markings

- `glColor3f(1.0f, 1.0f, 1.0f)`: Sets white color for markings
- `glPushMatrix()`: Saves matrix for each stripe
- `glTranslatef(0.0f, groundLevel - runwayThickness/2 + 0.01f, z)`: Positions slightly above runway
- `glScalef(stripeWidth, 0.01f, stripeLength)`: Scales stripe
- `glutSolidCube(1.0f)`: Draws stripe as thin rectangle
- `glPopMatrix()`: Restores matrix after each stripe

Grass Areas

- `glColor3f(0.0f, 0.4f, 0.0f)`: Sets green color
- `glPushMatrix()`: Saves matrix for grass areas
- `glTranslatef(position, groundLevel - runwayThickness/2, 0.0f)`: Positions grass
- `glScalef(grassWidth, runwayThickness*0.9f, runwayLength)`: Scales grass areas
- `glutSolidCube(1.0f)`: Draws grass as flat green rectangle
- `glPopMatrix()`: Restores matrix

7. HUD Drawing (drawHUD function)

- `glMatrixMode(GL_PROJECTION)`: Switches to projection matrix
- `glPushMatrix()`: Saves current projection
- `glLoadIdentity()`: Resets to identity
- `glOrtho(0.0, 800, 0.0, 600, -1.0, 1.0)`: Sets 2D orthographic projection
- `glMatrixMode(GL_MODELVIEW)`: Switches to modelview matrix
- `glPushMatrix()`: Saves current modelview
- `glLoadIdentity()`: Resets to identity
- `glColor3f(1.0f, 1.0f, 1.0f)`: Sets white text color
- `glRasterPos2i(x, y)`: Sets position for text rendering
- `glutBitmapCharacter(GLUT_BITMAP_9_BY_15, *c)`: Renders each character
- `glMatrixMode(GL_PROJECTION)`: Returns to projection matrix
- `glPopMatrix()`: Restores saved projection
- `glMatrixMode(GL_MODELVIEW)`: Returns to modelview matrix
- `glPopMatrix()`: Restores saved modelview

8. Game State Screens

Start Screen (drawStartScreen function)

- Matrix and projection manipulation as in HUD

- glColor3f(0.8f, 0.0f, 1.0f): Sets purple color for title
- glRasterPos2i(300, 460): Positions title text
- glutBitmapCharacter(GLUT_BITMAP_TIMES_ROMAN_24, *c): Renders title with larger font
- glColor3f(1.0f, 0.2f, 0.8f): Sets pink color for instructions
- Additional plane rendering with specific transformations

Control Select Screen (drawControlSelect function)

- Similar matrix and projection manipulation
- glColor3f(0.2f, 0.6f, 1.0f): Sets blue color for title
- Various text rendering with glRasterPos2i and glutBitmapCharacter
- glBegin(GL_QUADS): Draws mouse icon
- glVertex2f(x, y): Specifies 2D vertices
- glEnd(): Ends drawing
- glBegin(GL_LINE_LOOP): Draws keyboard outline

Game Over Screen (drawGameOver function)

- Similar matrix and projection manipulation
- glColor3f(1.0f, 0.0f, 0.0f): Sets red color for game over text
- Text rendering for score and instructions

9. Update and Reshape Functions

Update Function

- glutPostRedisplay(): Marks window for redrawing
- glutTimerFunc(16, update, 0): Schedules next update (60 FPS)

Reshape Function

- glViewport(0, 0, w, h): Adjusts viewport to window size
- glMatrixMode(GL_PROJECTION): Switches to projection matrix
- glLoadIdentity(): Resets projection
- gluPerspective(45.0f, (float)w / (float)h, 0.1f, 200.0f): Updates perspective with new aspect ratio
- glMatrixMode(GL_MODELVIEW): Switches back to modelview

- `glLoadIdentity():` Resets modelview

10. Input Handling Functions

Mouse Motion

- `glutWarpPointer(mouseX, mouseY):` Recenters mouse cursor
- `glutSetCursor(GLUT_CURSOR_NONE/INHERIT):` Controls cursor visibility

11. Lighting System Implementation Detail

Global Lighting Setup (in main)

- `glEnable(GL_LIGHTING):` Activates OpenGL's lighting system globally
- `glEnable(GL_LIGHT0):` Enables the first light source (OpenGL supports up to 8 lights, numbered 0-7)
- `GLfloat ambient[] = { 1.0f, 0.0f, 0.0f }:` Defines ambient light as red, affecting overall scene illumination
- `GLfloat diffuse[] = { 0.8f, 0.8f, 0.8f, 1.0f }:` Defines diffuse light as white with full opacity
- `glLightfv(GL_LIGHT0, GL_AMBIENT, ambient):` Applies ambient parameters to light 0
 - The 'fv' suffix indicates an array of floats is being passed
- `glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuse):` Applies diffuse parameters to light 0
- `glEnable(GL_NORMALIZE):` Ensures all normal vectors are normalized to unit length
 - Critical for accurate lighting calculations, especially when objects are scaled

Obstacle-Specific Lighting (in drawObstacles)

- `glEnable(GL_LIGHTING):` Ensures lighting is active for obstacles
- `glEnable(GL_LIGHT0):` Ensures light 0 is active
- `GLfloat lightPos[] = {0.0f, 10.0f, 10.0f, 1.0f}:` Positions light above and behind the camera
 - The fourth component (1.0f) makes this a positional light (not directional)
- `glLightfv(GL_LIGHT0, GL_POSITION, lightPos):` Updates light position
- `glEnable(GL_COLOR_MATERIAL):` Enables material color tracking
 - This simplifies material setup by allowing `glColor` calls to set material properties

- `glColorMaterial(GL_FRONT, GL_AMBIENT_AND_DIFFUSE)`: Specifies that `glColor` affects both ambient and diffuse material properties
 - `GL_FRONT` indicates only front-facing polygons are affected
- `glDisable(GL_COLOR_MATERIAL)`: Turns off material color tracking after obstacles are drawn
- `glDisable(GL_LIGHTING)`: Turns off lighting after obstacles are drawn
 - This is important for 2D HUD elements that shouldn't be affected by lighting

12. Advanced Matrix Stack Usage

Matrix Stack Management

- `glPushMatrix()`: Pushes the current matrix onto the stack
 - OpenGL maintains separate stacks for `MODELVIEW` and `PROJECTION` matrices
 - The stack depth is implementation-dependent but guaranteed to be at least 32 for `MODELVIEW`
- `glPopMatrix()`: Pops the top matrix from the stack, restoring the previous state
 - These operations allow preservation of transformation states for hierarchical modeling

Transformation Composition

- Transformations are applied in reverse order of specification due to matrix multiplication
- In `drawPlane()`:
 - `glTranslatef(planeX, planeY, -5.0f)`: Positions the plane in world space
 - `glRotatef(roll, 0, 0, 1)`: Applies roll rotation around Z-axis
 - `glRotatef(pitch, 1, 0, 0)`: Applies pitch rotation around X-axis
 - `glRotatef(90, 0, 1, 0)`: Applies yaw rotation to orient plane properly
 - `glScalef(0.8f, 0.8f, 0.8f)`: Scales the plane model
 - Actual transformation sequence: Scale → Yaw → Pitch → Roll → Translate

13. Custom Primitive Construction

Custom Plane Model

- Uses GL_TRIANGLES and GL_QUADS to construct a paper airplane model:
 - glBegin(GL_TRIANGLES): Begins triangle primitive definition
 - Used for nose, bottom, center crease, and tail fin
 - glBegin(GL_QUADS): Begins quadrilateral primitive definition
 - Used for left and right wings
 - glVertex3f(x, y, z): Specifies each vertex in 3D space
 - Vertices must be specified in counter-clockwise order for proper front-face determination
 - glEnd(): Ends the primitive definition

Vertex Specification Order

- For GL_TRIANGLES:
 - Every 3 vertices define one triangle

Example: Nose triangle is defined by:

```
c
glVertex3f(1.0f, 0.0f, 0.0f);    // Nose tip
glVertex3f(-1.0f, 0.2f, 0.5f);   // Top-left corner
                                // Top-right corner
                                glVertex3f(-1.0f, 0.2f, -0.5f);
```

- For GL_QUADS:
 - Every 4 vertices define one quadrilateral

Example: Left wing is defined by:

```
c
glVertex3f(-1.0f, 0.2f, 0.5f);   // Inner top
glVertex3f(-1.8f, -0.1f, 1.5f);  // Outer top
glVertex3f(-1.8f, -0.2f, 1.5f);  // Outer bottom
                                // Inner bottom
                                glVertex3f(-1.0f, -0.2f, 0.5f);
```

14. Text Rendering Mechanism

Text Rendering Process

- glRasterPos2i(x, y): Sets the raster position in window coordinates

- The raster position is transformed by the current modelview and projection matrices
- This function fails silently if the position would be clipped
- `glutBitmapCharacter(font, character)`: Renders a character using specified bitmap font
 - Used fonts include:
 - `GLUT_BITMAP_TIMES_ROMAN_24`: Larger serif font for titles
 - `GLUT_BITMAP_HELVETICA_18`: Medium sans-serif font for instructions
 - `GLUT_BITMAP_9_BY_15`: Monospace font for HUD elements
 - `GLUT_BITMAP_8_BY_13`: Smaller monospace font for additional information

2D Overlay Setup

- For HUD and game state screens:
 1. Save current projection matrix: `glMatrixMode(GL_PROJECTION); glPushMatrix();`
 2. Reset to identity: `glLoadIdentity();`
 3. Set orthographic projection: `glOrtho(0.0, 800, 0.0, 600, -1.0, 1.0);`
 - This creates a 2D coordinate system matching window pixels
 4. Save current modelview matrix: `glMatrixMode(GL_MODELVIEW); glPushMatrix();`
 5. Reset to identity: `glLoadIdentity();`
 6. Render text and 2D elements
 7. Restore previous matrices: `glPopMatrix()` for both `MODELVIEW` and `PROJECTION`

15. Viewport and Clipping Management

Viewport Configuration

- `glViewport(0, 0, w, h)`: Sets the viewport to match window dimensions
 - Parameters define lower-left corner (0,0) and width/height
 - This determines how normalized device coordinates map to window coordinates

Projection Matrix Configuration

- `gluPerspective(45.0f, (float)w / (float)h, 0.1f, 200.0f)`: Creates perspective projection
 - 45.0f: Field of view angle in degrees (vertical)

- (float)w / (float)h: Aspect ratio to prevent distortion
- 0.1f: Near clipping plane distance
- 200.0f: Far clipping plane distance
- Objects closer than 0.1 or farther than 200 units are clipped
- glOrtho(0.0, 800, 0.0, 600, -1.0, 1.0): Creates orthographic projection
 - Parameters define left, right, bottom, top, near, far clipping planes
 - Used for 2D rendering of HUD and menu screens

16. Camera Control Implementation

Camera Positioning

- gluLookAt(0.0f, planeY, 5.0f, 0.0f, planeY, -5.0f, 0.0f, 1.0f, 0.0f): Sets camera position
 - First three parameters: Camera position (eye point)
 - Next three parameters: Look-at point (center of view)
 - Last three parameters: Up vector (defines camera orientation)
 - This creates a camera that follows the plane vertically but maintains horizontal position

Camera-to-World Transformation

- The gluLookAt function creates a viewing transformation matrix that:
 1. Translates the scene so the eye point is at the origin
 2. Rotates the scene to align the direction vector with the negative z-axis
 3. Rotates the scene to align the up vector with the positive y-axis

17. Collision Detection and Boundary Constraints

Collision Detection Implementation

- Uses Euclidean distance calculation between plane and obstacle centers
- No specific OpenGL functions, but relies on the 3D coordinate system established by OpenGL

Boundary Constraint Implementation

Calculates visible frustum bounds at plane's z-position:

c

float z = 5.0f;

float fov = 45.0f;

float aspect = 800.0f / 600.0f;

```
float top = tan(fov * 0.5f * M_PI / 180.0f) * z;
```

- float right = top * aspect;
- Clamps plane position within these calculated bounds plus a margin

18. Advanced Rendering Techniques

Rotation Animation

- For obstacles: obstacles[i].rotation += 5.0f;
- If exceeding 360°: if (obstacles[i].rotation > 360.0f) obstacles[i].rotation -= 360.0f;
- Applied using: glRotatef(obstacles[i].rotation, 0.0f, 0.0f, 1.0f);

Building Window Rendering

Creates array of small dark cubes slightly offset from building surface:

```
c  
glPushMatrix();  
float winX = -width/4.0f + window * windowSpacing * 2;  
float winY = -height/2.0f + floor * windowSpacing + windowSpacing/2.0f;  
glTranslatef(winX, winY, width/2.0f + 0.01f);  
glScalef(windowSize, windowSize, 0.01f);  
glutSolidCube(1.0f);
```

- glPopMatrix();
- The 0.01f offset in z-direction prevents z-fighting with building surface

This expanded analysis provides deeper insight into the OpenGL functions used throughout the code, explaining not just what they do but how they work together to implement the game's visual features and interactive elements.

OUTPUT



